Design Document

Problem Understanding

Campus transport suffers when ride demand, driver availability, and passenger requests are coordinated through informal channels. This platform centralizes ride discovery, assignment, and status tracking for a campus-scale e-rickshaw system.

System Architecture

The application uses Next.js for both frontend and backend routes. MongoDB stores users, driver profiles, rides, ratings, scheduled bookings, and payment records. Socket.IO runs beside Next.js through `server.js` and broadcasts ride/driver updates to passengers and drivers.

Database Schema

- `User`: name, email, password hash, role, phone, department, year
- `DriverProfile`: user, vehicle number/type, license number, verification status, online status, current location
- `Ride`: passenger, driver, pickup, destination, status, fare, schedule time, timestamps
- `Rating`: ride, passenger, driver, score, feedback
- `Payment`: ride, amount, method, status, transaction reference

ERD

```
erDiagram
  User ||--o| DriverProfile : has
  User ||--o{ Ride : requests
  DriverProfile ||--o{ Ride : accepts
  Ride ||--o| Rating : receives
  Ride ||--o| Payment : has
```

API Overview

- `POST /api/auth/register`: create passenger or driver account
- `POST /api/auth/login`: create secure session
- `POST /api/auth/logout`: clear session
- `GET /api/auth/me`: return current user profile
- `PATCH /api/drivers/status`: update driver online/offline availability
- `GET /api/drivers/available`: list online verified drivers
- `GET/POST /api/rides`: list or create ride requests
- `PATCH /api/rides/[id]`: accept, reject, start, complete, or cancel rides

- `POST /api/ratings`: submit completed ride feedback
- `GET /api/analytics/demand`: return demand and dashboard statistics
- `PATCH /api/profile`: update passenger profile or driver vehicle/location details

Design Decisions

- HTTP-only JWT cookies are used so client code cannot forge sessions.
- Ride acceptance is checked server-side so one requested ride cannot be assigned to two drivers.
- Real-time events are emitted after successful database changes, keeping state synchronization tied to durable updates.
- Dashboard screens use dense cards, charts, and tables to support repeated operational use.
- Bonus features are scoped as practical campus-ready modules: Leaflet map markers, future ride scheduling input, simulated UPI/QR payment records, historical demand analytics, and simple demand forecasting from peak-hour history.